

ShadowFox Data Science Internship – Task 3

Advanced Level – Jupyter Notebook Data Analysis Project

E-Commerce Sales, Profitability & Customer Insights Analysis Using Python

Internship: ShadowFox Data Science Internship

Task: Task 3 – Advanced Level

Project Type: Jupyter Notebook Data Analysis Project

Tools: Python, Pandas, NumPy, Matplotlib, Seaborn

Dataset: Superstore Sales Dataset

Project Overview

This project focuses on analyzing e-commerce sales data using Python. The analysis explores sales performance, profitability, customer segments, product categories, regional performance, discounts, and sales trends. The aim is to identify meaningful patterns and generate data-driven business insights.

Objective

The main objective of this project is to explore and analyze the Superstore Sales dataset, identify important factors affecting sales and profit, and present meaningful insights through data analysis and visualizations.

1. Introduction

Data analysis plays an important role in understanding business performance and supporting better decision-making. E-commerce businesses generate large amounts of data related to customers, products, sales, discounts, and profits.

In this project, the Superstore Sales dataset is analyzed using Python. The dataset contains information about orders, customers, products, regions, sales, discounts, and profits.

The analysis will focus on identifying sales and profitability patterns across different categories, sub-categories, regions, customer segments, and time periods. Visualizations will be used to make the patterns easier to understand and interpret.

The project will conclude with important findings and business insights based on the analysis.

2. Research Questions

The following research questions will guide the analysis of the Superstore Sales dataset:

1. How do sales and profit change over time?
2. Which product categories and sub-categories generate the highest sales and profit?
3. How does sales and profitability vary across different regions?
4. Which customer segments contribute the most to sales and profit?
5. What is the relationship between discount and profit?
6. Which products or sub-categories have relatively low or negative profitability?
7. What patterns can be identified from the overall sales and profitability data?
8. What business insights and recommendations can be derived from the analysis?

3. Dataset Description

The dataset used for this project is the Superstore Sales Dataset. It contains transactional information about orders, customers, products, sales, discounts, and profits.

The dataset includes information such as:

- Order and shipping details
- Customer information

- Customer segments
- Geographic information
- Product categories and sub-categories
- Sales and quantity
- Discounts
- Profit

This dataset provides sufficient information to study sales performance and profitability from different business perspectives.

Key Variables

Variable	Description
Order Date	Date on which the order was placed
Ship Date	Date on which the order was shipped
Ship Mode	Shipping method used for the order
Segment	Customer segment
Region	Geographic sales region
Category	Main product category
Sub-Category	Product sub-category
Sales	Sales amount generated
Quantity	Number of products ordered
Discount	Discount provided on the order
Profit	Profit or loss generated

The dataset will first be examined for its structure, data types, missing values, and duplicate records before performing further analysis.

4. Importing Required Libraries

```

1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5
6 #Display settings
7 pd.set_option('display.max_columns', None)
8 pd.set_option('display.max_rows', 100)
9
10 #Set Visualization style
11 sns.set_style("whitegrid")
12
13 print("Libraries imported successfully")

```

Libraries imported successfully

5. Loading the Dataset

```

1 # Load the Superstore dataset
2
3 file_path=("/content/sample_-_superstore.xls")
4 df=pd.read_excel(file_path)
5
6 print("Dataset loaded successfully.")
7 print("Shape of the dataset:",df.shape)

```

Dataset loaded successfully.
Shape of the dataset: (10194, 21)

6. Dataset Structure

The Superstore Sales dataset contains 10,194 records and 21 columns. Each record represents an order-related transaction containing information about customers, products, sales, discounts, profits, shipping, and geographical details.

The structure of the dataset will be examined to understand the available variables and their data types before beginning the detailed analysis.

1 df.head()

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country/Region	City	State/Province	Postal Code	Region
0	1	US-2023-103800	2023-01-03	2023-01-07	Standard Class	DP-13000	Darren Powers	Consumer	United States	Houston	Texas	77095	Central
1	2	US-2023-112326	2023-01-04	2023-01-08	Standard Class	PO-19195	Phillina Ober	Home Office	United States	Naperville	Illinois	60540	Central
2	3	US-2023-112326	2023-01-04	2023-01-08	Standard Class	PO-19195	Phillina Ober	Home Office	United States	Naperville	Illinois	60540	Central
3	4	US-2023-112326	2023-01-04	2023-01-08	Standard Class	PO-19195	Phillina Ober	Home Office	United States	Naperville	Illinois	60540	Central
4	5	US-2023-141817	2023-01-05	2023-01-12	Standard Class	MB-18085	Mick Brown	Consumer	United States	Philadelphia	Pennsylvania	19143	East

7. Checking Column Names and Data Types

Understanding the column names and data types helps us identify the type of information available in the dataset and ensures that each variable is suitable for further analysis.

```
1 # Display column names and their data types
2
3 print("Column Names:")
4 print(df.columns.tolist())
5 print("\nData Types:")
6 print(df.dtypes)
```

```
Column Names:
['Row ID', 'Order ID', 'Order Date', 'Ship Date', 'Ship Mode', 'Customer ID', 'Customer Name', 'Segment', 'Country/Region', 'City', 'State/Province', 'Postal Code', 'Region']

Data Types:
Row ID          int64
Order ID        object
Order Date      datetime64[ns]
Ship Date       datetime64[ns]
Ship Mode       object
Customer ID     object
Customer Name   object
Segment         object
Country/Region  object
City            object
State/Province  object
Postal Code     object
Region          object
Product ID      object
Category        object
Sub-Category    object
Product Name    object
Sales           float64
Quantity        int64
Discount        float64
Profit          float64
dtype: object
```

8. Checking Missing Values

Missing values can affect the accuracy of data analysis. Therefore, the dataset is checked to identify whether any columns contain missing or null values.

```
1 # Check for missing values
2
3 missing_values = df.isnull().sum()
4
5 print("Missing values in each column:")
6 print(missing_values)
```

```
Missing values in each column:
Row ID      0
Order ID    0
Order Date  0
Ship Date   0
Ship Mode   0
Customer ID 0
Customer Name 0
Segment     0
Country/Region 0
City         0
State/Province 0
Postal Code  0
Region       0
Product ID   0
Category     0
Sub-Category 0
Product Name 0
Sales        0
Quantity     0
Discount     0
Profit       0
dtype: int64
```

9. Checking Duplicate Records

Duplicate records can affect the accuracy of analysis by counting the same transaction more than once. Therefore, the dataset is checked for duplicate rows before proceeding with further analysis.

```
1 # Check for duplicate records
2
3 duplicate_count=df.duplicated().sum()
4 print("Number of duplicate records:",duplicate_count)
```

```
Number of duplicate records: 0
```

10. Statistical Summary

A statistical summary is generated to understand the numerical variables in the dataset. It provides measures such as count, mean, standard deviation, minimum, maximum, and quartile values.

This helps us understand the general distribution and range of important numerical variables such as sales, quantity, discount, and profit.

```
1 df.describe()
```

	Row ID	Order Date	Ship Date	Sales	Quantity	Discount	Profit
count	10194.000000	10194	10194	10194.000000	10194.000000	10194.000000	10194.000000
mean	5097.500000	2025-04-29 11:48:25.002942720	2025-05-03 10:52:45.626839296	228.225854	3.791838	0.155385	28.673417
min	1.000000	2023-01-03 00:00:00	2023-01-07 00:00:00	0.444000	1.000000	0.000000	-6599.978000
25%	2549.250000	2024-05-14 00:00:00	2024-05-19 00:00:00	17.220000	2.000000	0.000000	1.760800
50%	5097.500000	2025-06-25 00:00:00	2025-06-28 00:00:00	53.910000	3.000000	0.200000	8.690000
75%	7645.750000	2026-05-14 00:00:00	2026-05-18 00:00:00	209.500000	5.000000	0.200000	29.297925
max	10194.000000	2026-12-30 00:00:00	2027-01-05 00:00:00	22638.480000	14.000000	0.800000	8399.976000

11. Converting Date Columns

The Order Date and Ship Date columns are converted into date format. This allows us to perform time-based analysis, such as studying sales and profit trends over different periods.

```
1 # Convert date columns to datetime format
2
3 df['Order Date'] = pd.to_datetime(df['Order Date'])
4 df['Ship Date'] = pd.to_datetime(df['Ship Date'])
5
6 # Check the updated data types
7
8 print("Date columns converted successfully.\n")
9 print(df[['Order Date', 'Ship Date']].dtypes)
```

Date columns converted successfully.

```
Order Date    datetime64[ns]
Ship Date     datetime64[ns]
dtype: object
```

12. Creating Time-Based Features

To perform time-based analysis, additional features are created from the Order Date column. Year, Month, and Year-Month values will help us examine changes in sales and profit over time.

```
1 # Create time-based features
2
3 df['Year'] = df['Order Date'].dt.year
4 df['Month'] = df['Order Date'].dt.month
5 df['Year-Month'] = df['Order Date'].dt.to_period('M').astype(str)
6
7 # Display the new columns
8
9 df[['Order Date', 'Year', 'Month', 'Year-Month']].head()
```

	Order Date	Year	Month	Year-Month
0	2023-01-03	2023	1	2023-01
1	2023-01-04	2023	1	2023-01
2	2023-01-04	2023	1	2023-01
3	2023-01-04	2023	1	2023-01
4	2023-01-05	2023	1	2023-01

13. Sales and Profit Trends Over Time

Analyzing sales and profit over time helps identify changes in business performance across different periods. Monthly sales and profit will be calculated and visualized to observe overall trends.

```
1 # Calculate monthly sales and profit
2
3 monthly_performance = df.groupby('Year-Month')[['Sales', 'Profit']].sum().reset_index()
4
5 monthly_performance
```

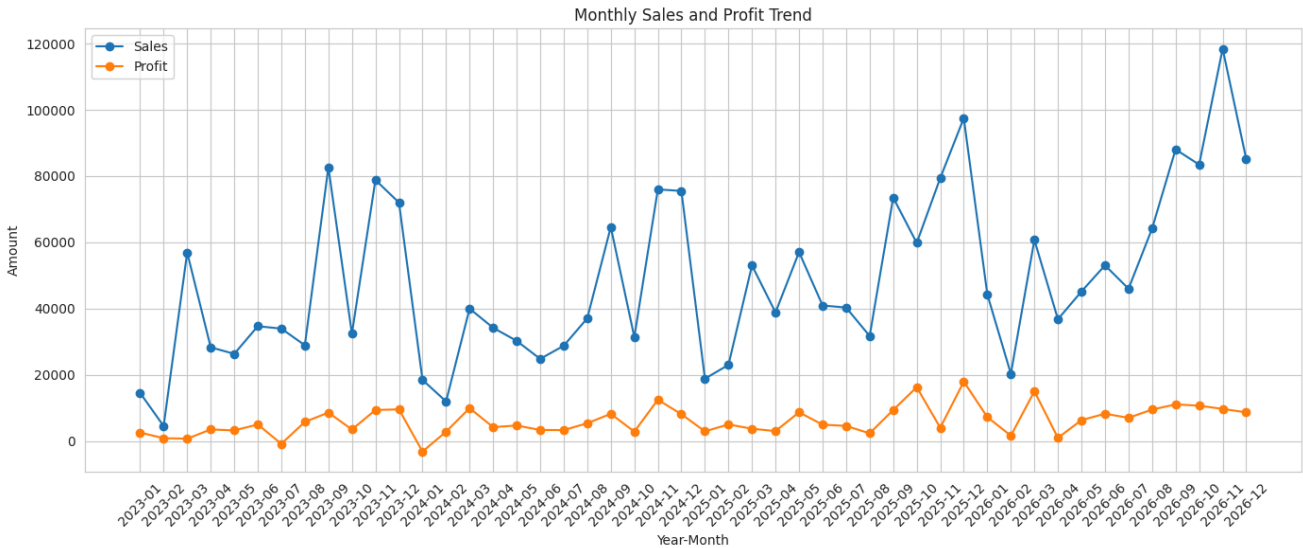
Next steps:

[Generate code with monthly_performance](#)[New interactive sheet](#)

	Year-Month	Sales	Profit
0	2023-01	14518.0550	2539.3907
1	2023-02	4519.8920	862.3084

14. Monthly Sales and Profit Trend

```
1 # Visualize monthly sales and profit
2
3 plt.figure(figsize=(14, 6))
4
5 plt.plot(
6     monthly_performance['Year-Month'],
7     monthly_performance['Sales'],
8     marker='o',
9     label='Sales'
10 )
11
12 plt.plot(
13     monthly_performance['Year-Month'],
14     monthly_performance['Profit'],
15     marker='o',
16     label='Profit'
17 )
18
19 plt.title('Monthly Sales and Profit Trend')
20 plt.xlabel('Year-Month')
21 plt.ylabel('Amount')
22 plt.xticks(rotation=45)
23 plt.legend()
24 plt.tight_layout()
25 plt.show()
```



33	2025-10	59831.0010	16256.8422
34	2025-11	79411.9658	4011.4075
35	2025-12	97502.2770	17926.3013
36	2026-01	44259.2140	7231.6391
37	2026-02	20301.1334	1613.8720
38	2026-03	60728.4808	15013.0910

15. Interpretation of Sales and Profit Trends

40	2026-05	45155.4822	6299.8072
41	2026-06	53056.0777	8246.5707
42	2026-07	45989.4960	7006.5012
43	2026-08	64129.7580	9488.0722
44	2026-09	88064.5320	11050.8024
45	2026-10	83474.7832	10670.5312

The monthly analysis provides an overview of how sales and profit change across different periods. The visualization helps identify months with relatively higher or lower business performance and shows whether changes in sales are accompanied by similar changes in profit. The sales values are generally higher than the profit values because profit represents the amount remaining after business costs are considered. The trend also helps identify periods where sales increase but profit does not increase at the same rate, which may indicate the influence of discounts, product mix, or other business factors.

Overall, the monthly trend provides a useful foundation for understanding changes in the company's sales and profitability over time.

46 2026-11 118454.5050 9692.1037

47 2026-12 85175.0328 8655.8269

16. Category-wise Sales and Profit Analysis

Product categories can have different levels of sales and profitability. In this section, sales and profit are grouped by category to compare the overall performance of each product category.

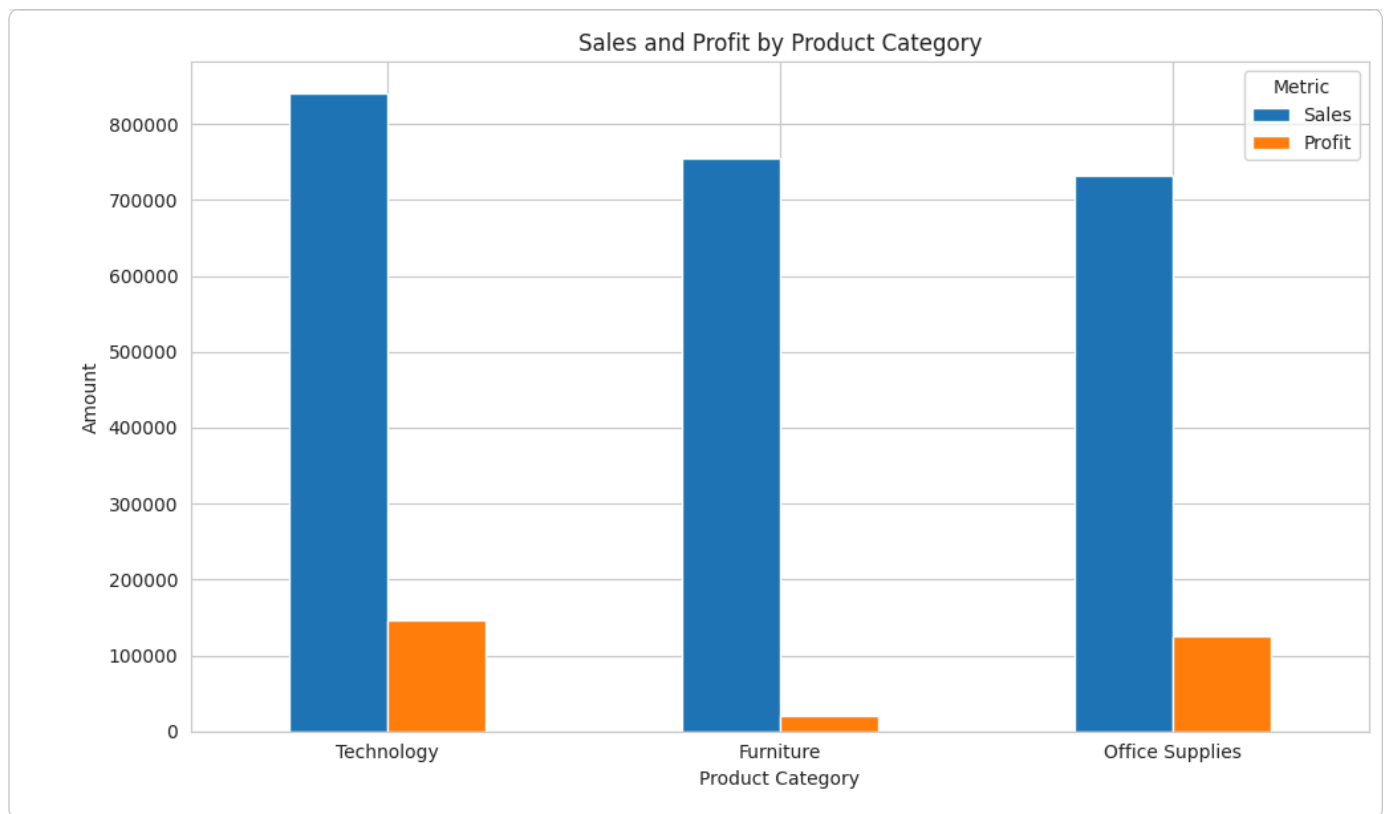
```
1 # Calculate sales and profit by category
2
3 category_performance = (
4     df.groupby('Category')[['Sales', 'Profit']]
5     .sum()
6     .sort_values('Sales', ascending=False)
7 )
8
9 category_performance
```

	Sales	Profit	
Category			
Technology	839893.2790	146543.3756	
Furniture	754747.7613	19729.9956	
Office Supplies	731893.3140	126023.4434	

Next steps: [Generate code with category_performance](#) [New interactive sheet](#)

17. Category-wise Sales and Profit Visualization

```
1 # Visualize sales and profit by category
2
3 category_performance.plot(
4     kind='bar',
5     figsize=(10, 6)
6 )
7
8 plt.title('Sales and Profit by Product Category')
9 plt.xlabel('Product Category')
10 plt.ylabel('Amount')
11 plt.xticks(rotation=0)
12 plt.legend(title='Metric')
13 plt.tight_layout()
14 plt.show()
```



18. Interpretation of Category-wise Sales and Profit

The category-wise analysis shows differences in sales and profitability across the product categories.

The comparison helps identify which categories contribute more to overall sales and which categories generate stronger or weaker profit. A category with high sales does not necessarily have the highest profit, because profitability can also be affected by discounts, product costs, and the type of products sold.

This analysis provides a useful understanding of the contribution of each product category to the overall business performance.

✓ 19. Sub-Category-wise Sales and Profit Analysis

Sub-categories provide a more detailed view of product performance. Analyzing sales and profit at the sub-category level helps identify products that contribute strongly to revenue and those that may require further attention because of lower profitability.

```
1 # Calculate sales and profit by sub-category
2
3 subcategory_performance = (
4     df.groupby('Sub-Category')[['Sales', 'Profit']]
5     .sum()
6     .sort_values('Sales', ascending=False)
7 )
8
9 subcategory_performance
```

	Sales	Profit	
Sub-Category			
Chairs	335768.2490	27223.5323	
Phones	331842.6400	45050.8265	
Storage	224644.5540	21285.1115	
Tables	208020.1820	-17753.2061	
Binders	207354.8810	31426.1003	
Machines	189925.0310	3461.9769	
Accessories	167380.3180	41936.6357	
Copiers	150745.2900	56093.9365	
Bookcases	115361.2043	-3632.0736	
Appliances	108213.1850	18329.4844	
Furnishings	95598.1260	13891.7430	
Paper	79540.5380	34511.5070	
Supplies	46725.4980	-1171.3945	
Art	27659.0140	6653.1962	
Envelopes	16528.3620	6988.0247	
Labels	12695.0420	5572.7780	
Fasteners	8532.2400	2428.6358	

Next steps:

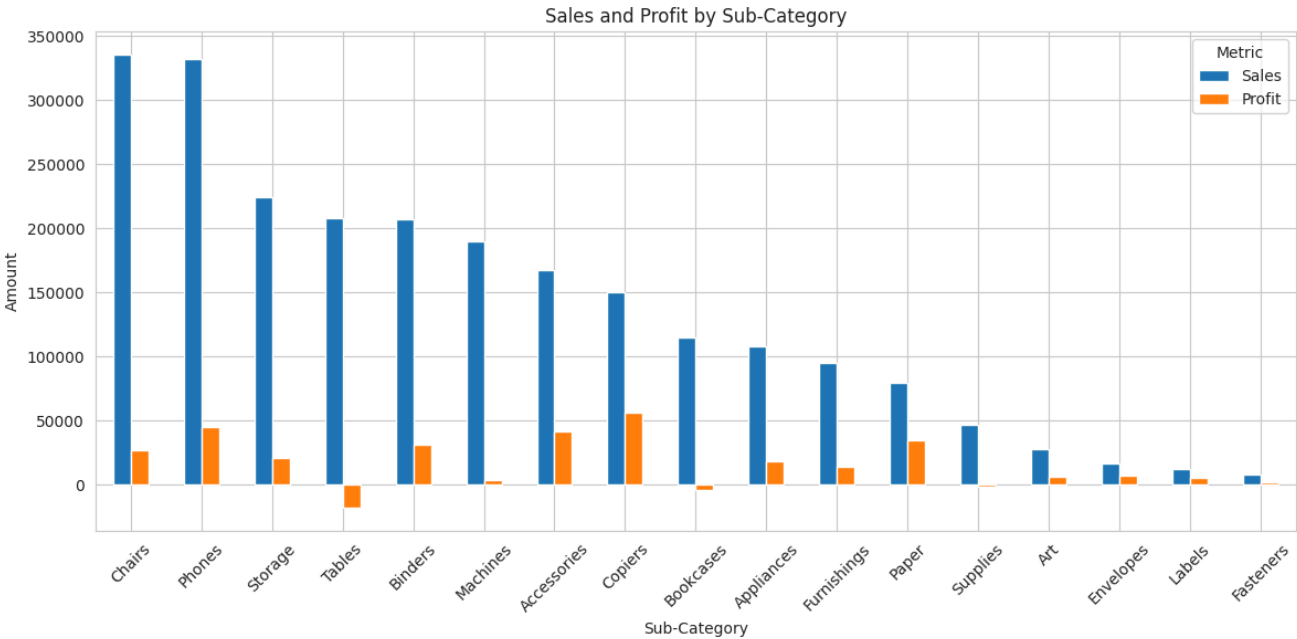
[Generate code with subcategory_performance](#)[New interactive sheet](#)

20. Sub-Category-wise Sales and Profit Visualization

```

1 # Visualize sales and profit by sub-category
2
3 subcategory_performance.plot(
4     kind='bar',
5     figsize=(12, 6)
6 )
7
8 plt.title('Sales and Profit by Sub-Category')
9 plt.xlabel('Sub-Category')
10 plt.ylabel('Amount')
11 plt.xticks(rotation=45)
12 plt.legend(title='Metric')
13 plt.tight_layout()
14 plt.show()

```



21. Interpretation of Sub-Category Performance

The sub-category analysis provides a more detailed view of product performance within the different categories. The results show that sales and profit contributions vary across sub-categories.

Some sub-categories contribute strongly to sales as well as profit, while others show a difference between their sales contribution and profitability. This indicates that high sales volume does not always result in proportionally high profit.

The comparison helps identify strong-performing areas as well as sub-categories that may need further investigation based on their profitability and sales performance.

22. Regional Sales and Profit Analysis

Geographical performance can vary across different regions. Analyzing sales and profit by region helps us understand how different areas contribute to the overall business performance.

```
1 # Calculate sales and profit by region
2
3 region_performance = (
4     df.groupby('Region')[['Sales', 'Profit']]
5     .sum()
6     .sort_values('Sales', ascending=False)
7 )
8
9 region_performance
```

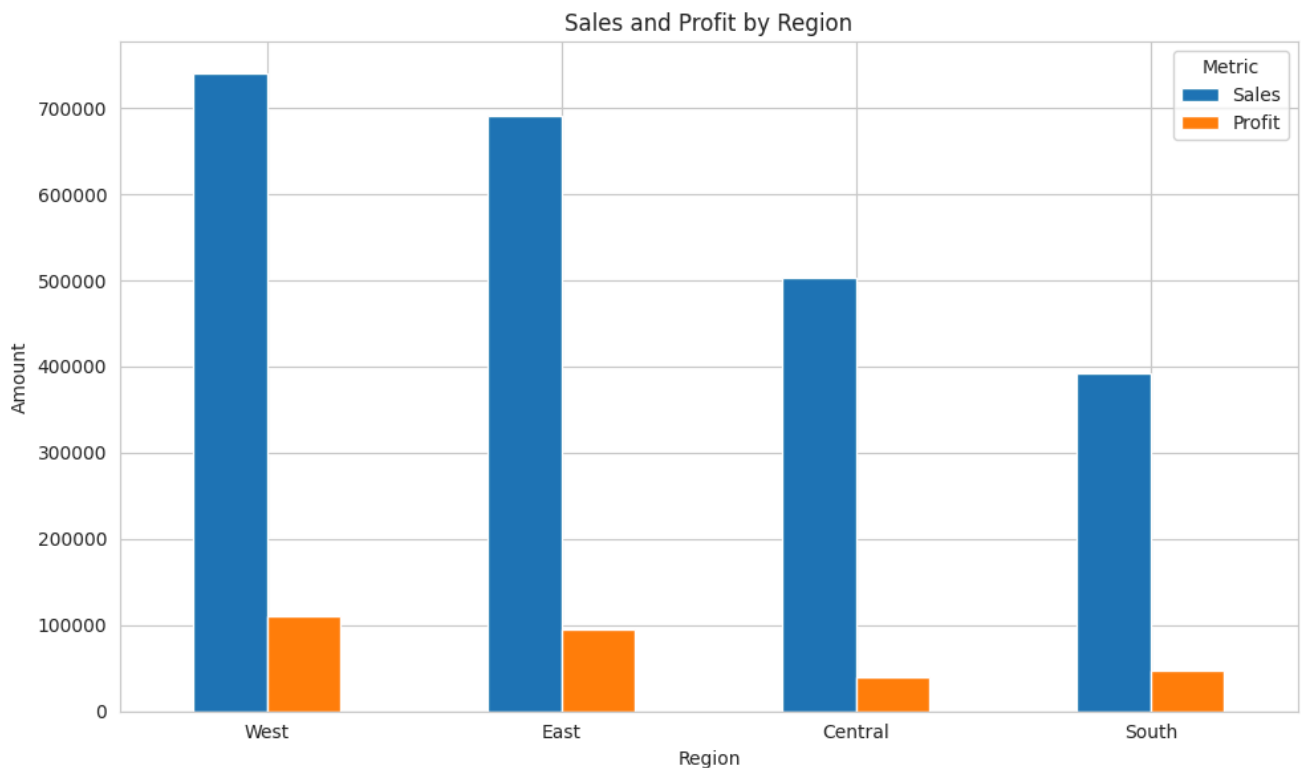
	Sales	Profit
Region		
West	739813.6085	110798.8170
East	691828.1680	94883.2603
Central	503170.6728	39865.3070
South	391721.9050	46749.4303

23. Regional Sales and Profit Visualization

```

1 # Visualize sales and profit by region
2
3 region_performance.plot(
4     kind='bar',
5     figsize=(10, 6)
6 )
7
8 plt.title('Sales and Profit by Region')
9 plt.xlabel('Region')
10 plt.ylabel('Amount')
11 plt.xticks(rotation=0)
12 plt.legend(title='Metric')
13 plt.tight_layout()
14 plt.show()

```



24. Interpretation of Regional Sales and Profit

The regional analysis shows that sales and profit contributions are not the same across all regions. Some regions contribute a larger share of total sales, while differences can also be observed in their profitability.

This comparison helps identify regions with stronger business performance and regions where profitability may require further investigation. The differences may be influenced by customer demand, product mix, pricing, discounts, and other regional business factors.

Overall, regional analysis provides useful insights into the geographical distribution of sales and profitability.

25. Customer Segment Sales and Profit Analysis

Customer segmentation helps us understand how different groups of customers contribute to the business. In this analysis, sales and profit will be compared across the available customer segments.

```

1 # Calculate sales and profit by customer segment
2
3 segment_performance = (
4     df.groupby('Segment')[['Sales', 'Profit']]
5     .sum()
6 )

```

```

5     .sum()
6     .sort_values('Sales', ascending=False)
7 )
8
9 segment_performance

```

	Sales	Profit
Segment		
Consumer	1.170660e+06	136371.4463
Corporate	7.158061e+05	94249.6400
Home Office	4.400684e+05	61675.7283

Next steps: [Generate code with segment_performance](#) [New interactive sheet](#)

26. Customer Segment Sales and Profit Visualization

```

1 # Visualize sales and profit by customer segment
2
3 segment_performance.plot(
4     kind='bar',
5     figsize=(10, 6)
6 )
7
8 plt.title('Sales and Profit by Customer Segment')
9 plt.xlabel('Customer Segment')
10 plt.ylabel('Amount')
11 plt.xticks(rotation=0)
12 plt.legend(title='Metric')
13 plt.tight_layout()
14 plt.show()

```



27. Interpretation of Customer Segment Performance

The customer segment analysis shows that the contribution to sales and profit differs across the customer segments.

The comparison helps identify which customer groups contribute more to overall business performance. It also shows that sales contribution and profit contribution may not always follow exactly the same pattern.

Understanding customer segment performance can help businesses focus on customer groups that generate meaningful sales and profitability while also examining areas where performance can be improved.

28. Discount vs Profit Analysis

Discounts can influence the profitability of products and orders. In this section, the relationship between discount levels and profit will be examined to understand whether changes in discount are associated with changes in profitability.

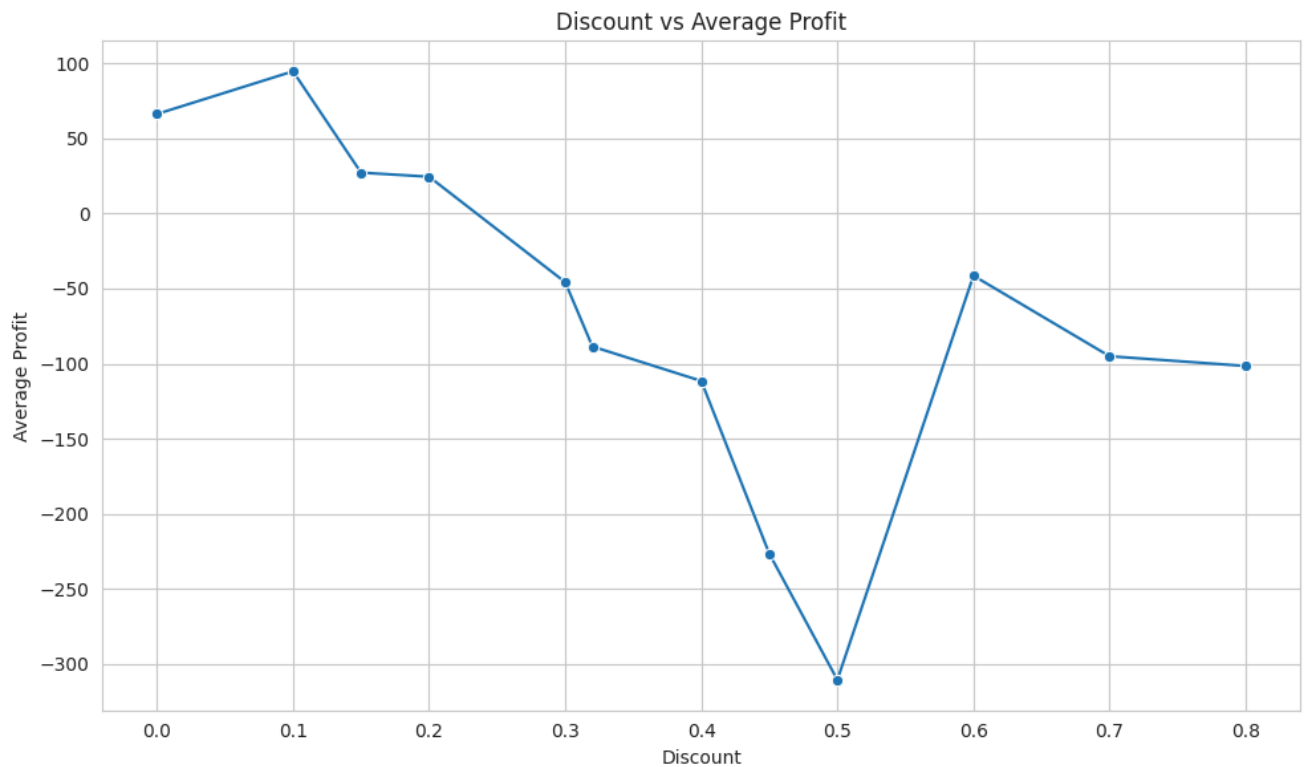
```
1 # Calculate average profit for each discount level
2
3 discount_profit = (
4     df.groupby('Discount')['Profit']
5     .mean()
6     .reset_index()
7 )
8
9 discount_profit
```

	Discount	Profit	
0	0.00	66.338799	
1	0.10	94.791354	
2	0.15	27.288298	
3	0.20	24.576350	
4	0.30	-45.710633	
5	0.32	-88.560656	
6	0.40	-111.528378	
7	0.45	-226.646464	
8	0.50	-310.703456	
9	0.60	-41.371458	
10	0.70	-95.048521	
11	0.80	-101.545745	

Next steps: [Generate code with discount_profit](#) [New interactive sheet](#)

29. Discount vs Average Profit Visualization

```
1 # Visualize the relationship between discount and average profit
2
3 plt.figure(figsize=(10, 6))
4
5 sns.lineplot(
6     data=discount_profit,
7     x='Discount',
8     y='Profit',
9     marker='o'
10 )
11
12 plt.title('Discount vs Average Profit')
13 plt.xlabel('Discount')
14 plt.ylabel('Average Profit')
15 plt.tight_layout()
16 plt.show()
```



30. Interpretation of Discount vs Average Profit

The analysis shows how average profit changes across different discount levels. Comparing discount levels with average profit helps us understand the possible impact of discounts on profitability.

The visualization can be used to identify discount levels associated with relatively higher or lower average profit. Higher discounts may reduce the amount of profit earned on individual sales, although the overall effect can also depend on factors such as product category, sales volume, and product pricing.

Therefore, discount levels should be considered carefully when evaluating profitability.

31. Top and Bottom Performing Sub-Categories

Identifying the highest and lowest performing sub-categories helps us understand which product areas contribute strongly to sales and profitability. This analysis can also highlight sub-categories that may require further investigation.

```
1 # Display top 5 and bottom 5 sub-categories based on profit
2
3 top_profit_subcategories = (
4     df.groupby('Sub-Category')['Profit']
5     .sum()
6     .sort_values(ascending=False)
7 )
8
9 print("Top 5 Sub-Categories by Profit:")
10 display(top_profit_subcategories.head(5))
11
12 print("\nBottom 5 Sub-Categories by Profit:")
13 display(top_profit_subcategories.tail(5))
```

Top 5 Sub-Categories by Profit:

Profit	
Sub-Category	
Copiers	56093.9365
Phones	45050.8265
Accessories	41936.6357
Paper	34511.5070
Binders	31426.1003

dtype: float64

Bottom 5 Sub-Categories by Profit:

Profit	
Sub-Category	
Machines	3461.9769
Fasteners	2428.6358
Supplies	-1171.3945
Bookcases	-3632.0736
Tables	-17753.2061

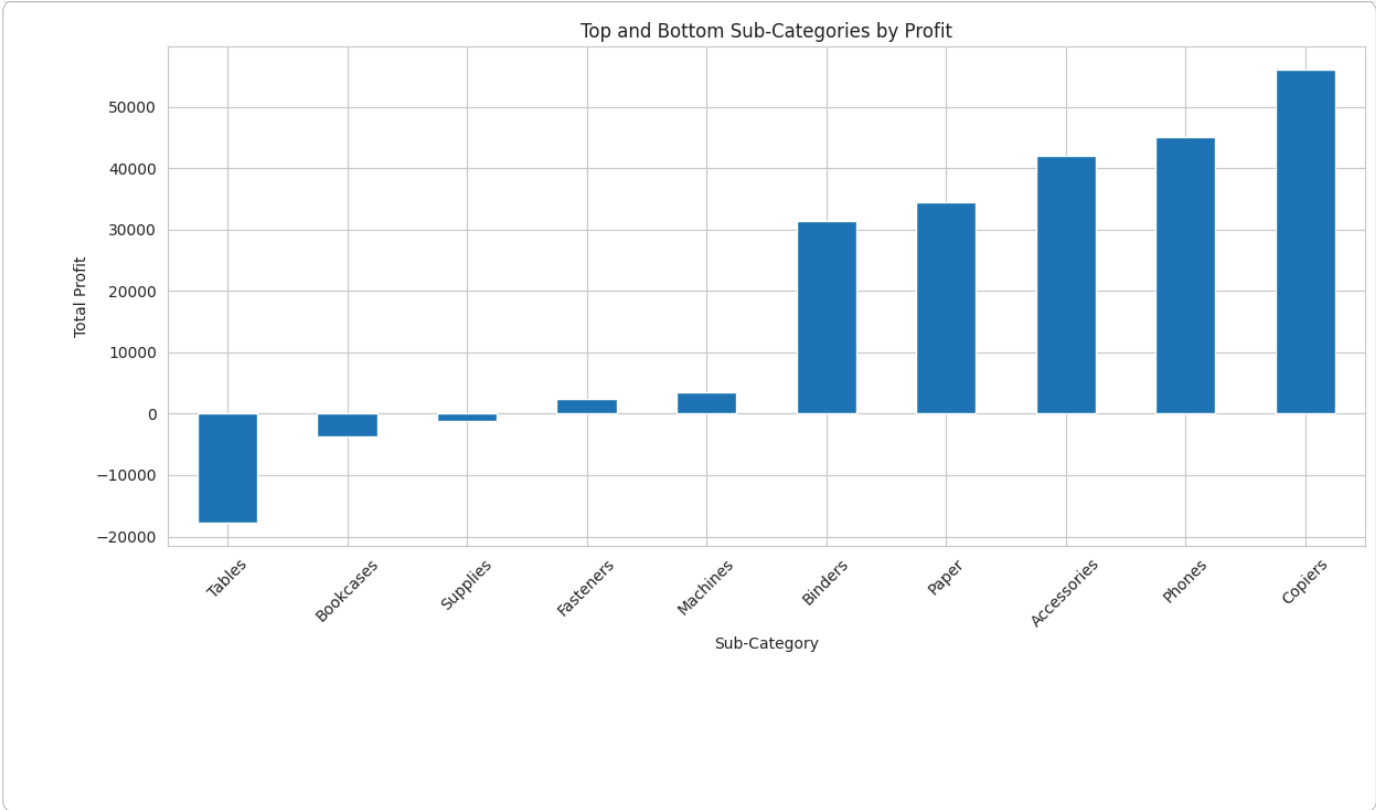
dtype: float64

✓ Top and Bottom Performing Sub-Categories Visualization

```

1 # Select top 5 and bottom 5 sub-categories based on total profit
2
3 selected_subcategories = pd.concat([
4     top_profit_subcategories.head(5),
5     top_profit_subcategories.tail(5)
6 ])
7
8 # Visualize the selected sub-categories
9
10 plt.figure(figsize=(12, 6))
11
12 selected_subcategories.sort_values().plot(kind='bar')
13
14 plt.title('Top and Bottom Sub-Categories by Profit')
15 plt.xlabel('Sub-Category')
16 plt.ylabel('Total Profit')
17 plt.xticks(rotation=45)
18 plt.tight_layout()
19 plt.show()

```



33. Interpretation of Top and Bottom Performing Sub-Categories

The analysis highlights differences in profitability among the various sub-categories. The top-performing sub-categories contribute the highest total profit, while the bottom-performing sub-categories contribute relatively less profit or may show losses.

This comparison helps identify product areas that contribute positively to business profitability and areas that may require further investigation. Factors such as discounts, sales volume, product pricing, and operating costs may influence these differences.

The results can support further analysis and help in developing appropriate business strategies.

34. Correlation Analysis

Correlation analysis is used to understand the strength and direction of relationships between numerical variables. In this project, the relationships among Sales, Quantity, Discount, and Profit are examined.

A correlation value closer to +1 indicates a strong positive relationship, while a value closer to -1 indicates a strong negative relationship. A value close to 0 indicates a weak linear relationship.

```
1 # Select important numerical variables
2
3 correlation_data = df[['Sales', 'Quantity', 'Discount', 'Profit']]
4
5 # Calculate the correlation matrix
6
7 correlation_matrix = correlation_data.corr()
8
9 correlation_matrix
```

	Sales	Quantity	Discount	Profit
Sales	1.000000	0.198457	-0.027756	0.481460
Quantity	0.198457	1.000000	0.007475	0.066145
Discount	-0.027756	0.007475	1.000000	-0.218882
Profit	0.481460	0.066145	-0.218882	1.000000

Next steps:

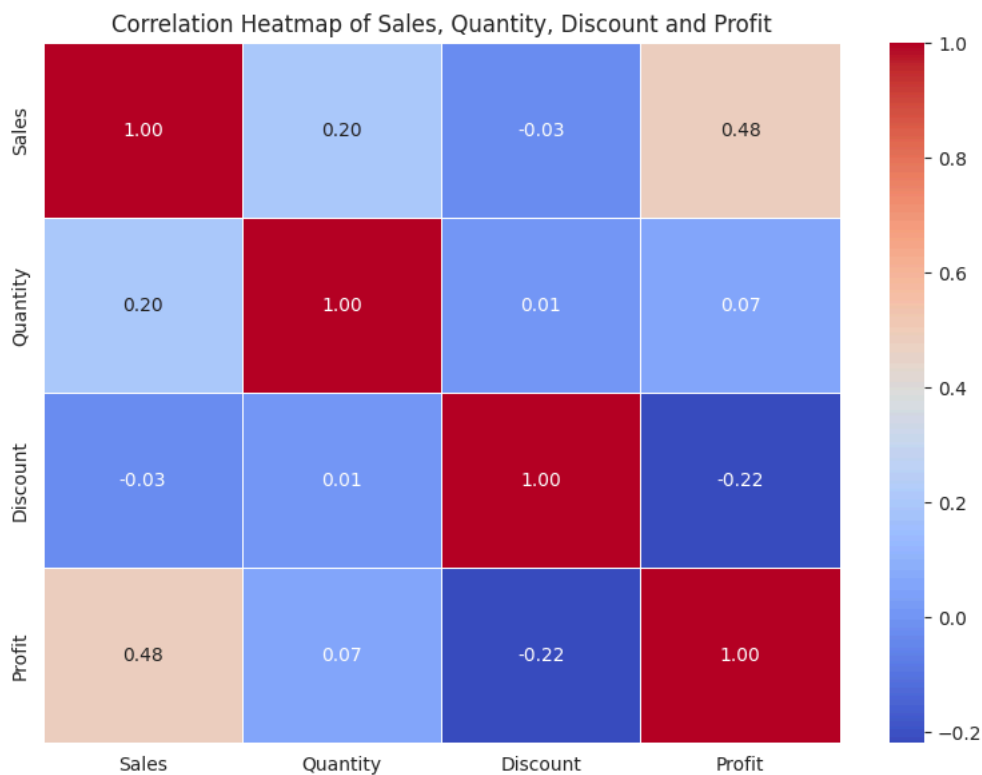
Generate code with correlation_matrix

New interactive sheet

35. Correlation Heatmap

A heatmap is used to visualize the correlation matrix. It makes it easier to identify positive and negative relationships between the numerical variables.

```
1 # Create a correlation heatmap
2
3 plt.figure(figsize=(8, 6))
4
5 sns.heatmap(
6     correlation_matrix,
7     annot=True,
8     fmt='.2f',
9     cmap='coolwarm',
10    linewidths=0.5
11 )
12
13 plt.title('Correlation Heatmap of Sales, Quantity, Discount and Profit')
14 plt.tight_layout()
15 plt.show()
```



36. Interpretation of Correlation Analysis

The correlation analysis helps us understand the linear relationships among Sales, Quantity, Discount, and Profit.

The heatmap makes it easier to identify variables that move together positively or negatively. A positive correlation indicates that two variables tend to increase together, while a negative correlation indicates that one variable tends to decrease as the other increases.

The correlation values should be interpreted as relationships rather than proof of cause and effect. Other business factors may also influence sales and profitability.

37. Year-wise Sales and Profit Analysis

Analyzing sales and profit by year helps us understand the overall growth and changes in business performance across different years. This analysis provides a broader view of the company's yearly sales and profitability.

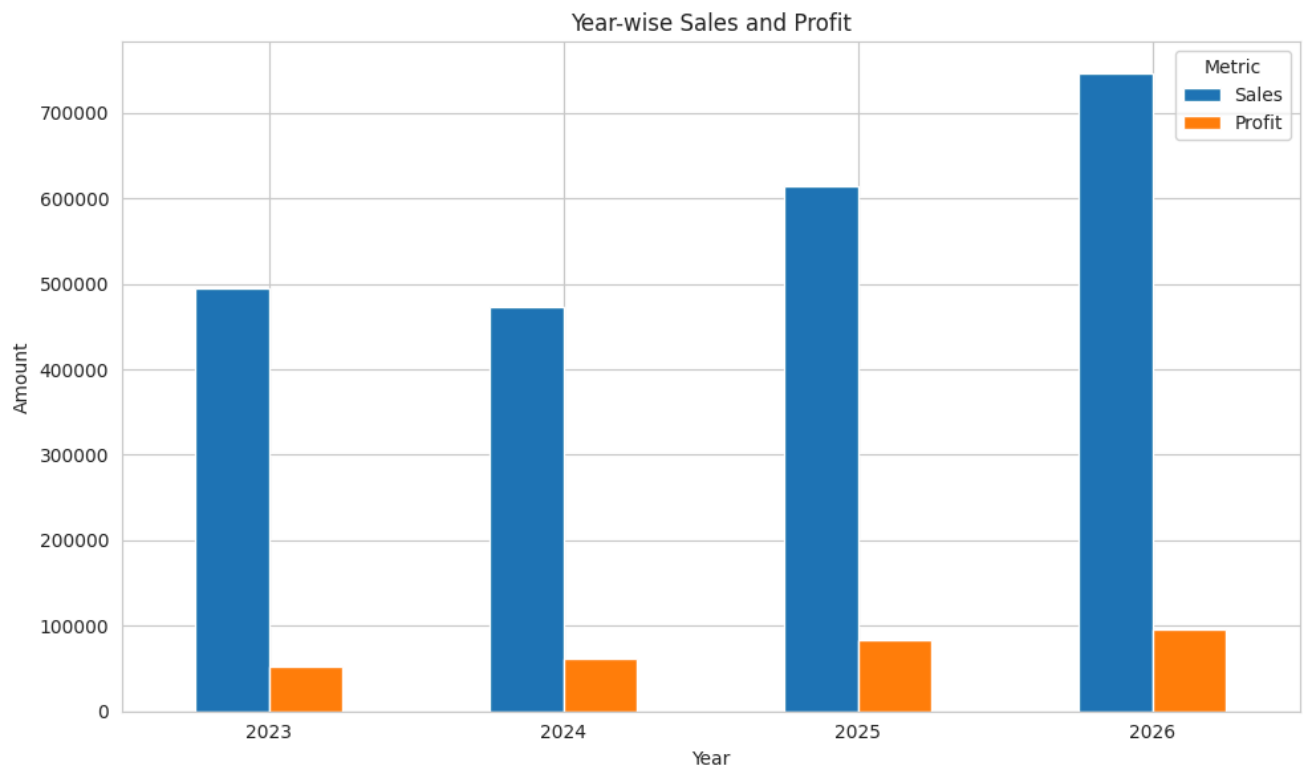
```
1 # Calculate total sales and profit by year
2
3 yearly_performance = (
4     df.groupby('Year')[['Sales', 'Profit']]
5     .sum()
6 )
7
8 yearly_performance
```

	Sales	Profit
Year		
2023	494040.2121	51684.2957
2024	472993.0310	62020.9695
2025	613933.5800	82665.2018
2026	745567.5312	95926.3476

Next steps: [Generate code with yearly_performance](#) [New interactive sheet](#)

38. Year-wise Sales and Profit Visualization

```
1 # Visualize yearly sales and profit
2
3 yearly_performance.plot(
4     kind='bar',
5     figsize=(10, 6)
6 )
7
8 plt.title('Year-wise Sales and Profit')
9 plt.xlabel('Year')
10 plt.ylabel('Amount')
11 plt.xticks(rotation=0)
12 plt.legend(title='Metric')
13 plt.tight_layout()
14 plt.show()
```



39. Interpretation of Year-wise Sales and Profit

The year-wise analysis provides an overview of how the business performed across different years. The comparison of total sales and profit helps identify changes in overall business performance over time.

The results show that sales and profit can vary from year to year. Differences in product demand, customer purchasing patterns, discounts, and other business factors can contribute to these changes.

Year-wise analysis is useful for understanding the overall direction of the business and provides a basis for further investigation of specific periods.

✓ 40. Key Findings

The key findings from the analysis will be summarized based on the actual results obtained from the dataset. The findings will cover sales and profit trends, category and sub-category performance, regional performance, customer segments, discount patterns, and relationships between numerical variables.

```

1 # Generate key summary values from the analysis
2
3 print("Overall Sales:", df['Sales'].sum())
4 print("Overall Profit:", df['Profit'].sum())
5
6 print("\nHighest Sales Category:")
7 print(category_performance['Sales'].idxmax(), "-", category_performance['Sales'].max())
8
9 print("\nHighest Profit Category:")
10 print(category_performance['Profit'].idxmax(), "-", category_performance['Profit'].max())
11
12 print("\nHighest Sales Region:")
13 print(region_performance['Sales'].idxmax(), "-", region_performance['Sales'].max())
14
15 print("\nHighest Profit Region:")
16 print(region_performance['Profit'].idxmax(), "-", region_performance['Profit'].max())
17
18 print("\nHighest Profit Customer Segment:")
19 print(segment_performance['Profit'].idxmax(), "-", segment_performance['Profit'].max())
20
21 print("\nMost Profitable Sub-Category:")
22 print(top_profit_subcategories.idxmax(), "-", top_profit_subcategories.max())
23
24 print("\nLeast Profitable Sub-Category:")
25 print(top_profit_subcategories.idxmin(), "-", top_profit_subcategories.min())

```

Overall Sales: 2326534.3543
Overall Profit: 292296.8145999999

Highest Sales Category:
Technology - 839893.279

Highest Profit Category:
Technology - 146543.37559999997

Highest Sales Region:
West - 739813.6085

Highest Profit Region:
West - 110798.81699999998

Highest Profit Customer Segment:
Consumer - 136371.44629999995

Most Profitable Sub-Category:
Copiers - 56093.936499999996

Least Profitable Sub-Category:
Tables - -17753.206100000007

40. Key Findings

The analysis of the Superstore Sales dataset produced the following key findings:

1. Overall Business Performance:

The dataset recorded total sales of approximately **2.33 million** and total profit of approximately **292,297**.